

Reporte de Arquitectura: AntiTurismo Puerto Montt

Benjamin Alberto Martinez Hernandez
Andres Fernando Hernandez Perez

17-07-2026

a) Contexto de negocio

AntiTurismo Puerto Montt es una plataforma web que promueve lugares poco convencionales de Puerto Montt, ofreciendo una alternativa al turismo tradicional. La aplicacion permite a los visitantes explorar recorridos tematicos (Marzo, Abril, etc.), enviar testimonios con fotos, y reservar cupos en buses de recorrido. Un administrador gestiona los lugares, modera los testimonios enviados por usuarios, y aprueba o rechaza contenido.

Los flujos principales son:

1. **Visitante:** explora lugares → envia testimonio con foto → reserva cupo en bus.
2. **Administrador:** crea/edita/elimina lugares → aprueba/rechaza testimonios.

La solucion esta completamente desplegada en AWS usando Infraestructura como Codigo (Terraform), con contenedores para el frontend y funciones serverless para el backend.

b) Diagrama de infraestructura

c) Justificacion tecnica

Proveedor y version de Terraform

Se utiliza Terraform con el backend remoto en S3 para almacenar el estado de forma colaborativa y segura. El proveedor AWS esta fijado a la version ~> 6.52.0 para garantizar estabilidad.

```
terraform {  
  required_version = ">= 1.15"  
  required_providers {  
    aws = {
```

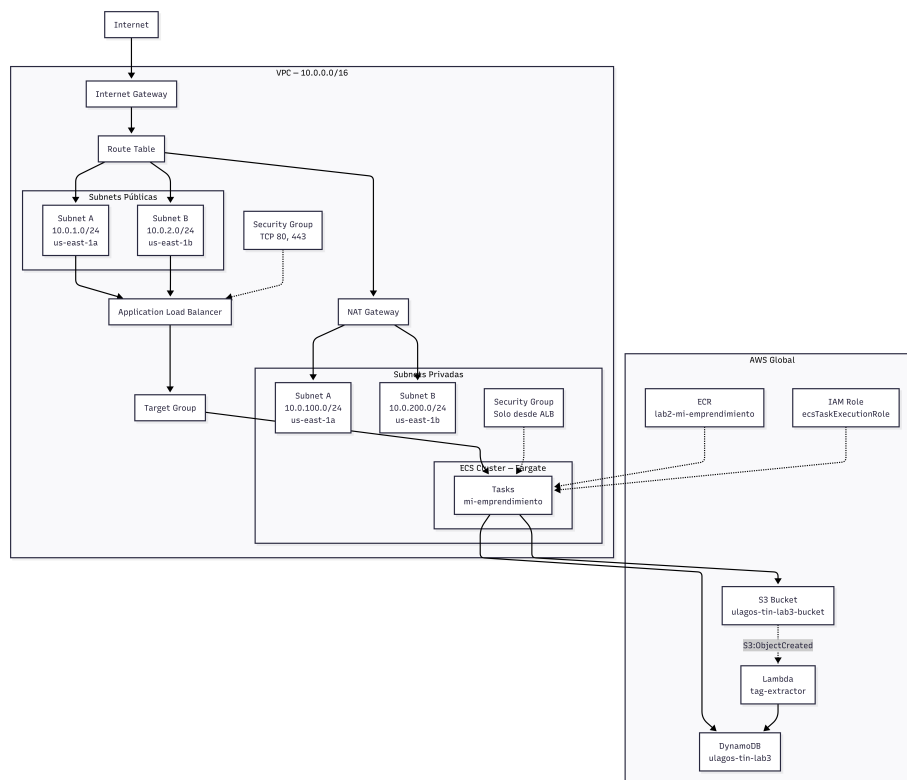


Figura 1: Diagrama de la infraestructura

```

        source = "hashicorp/aws"
        version = "~> 6.52.0"
    }
}
backend "s3" {}
}

```

- **Decision:** backend S3 con encriptacion habilitada permite compartir estado entre desarrolladores y evita conflictos de **terraform apply** simultaneo. No se uso DynamoDB para locking porque Terraform >= 1.15 elimino ese requisito con el nuevo mecanismo de **use_lockfile**.

VPC y Subnets

La VPC se crea con el modulo oficial **terraform-aws-modules/vpc/aws** ~> 6.6.1 usando el bloque CIDR 10.0.0.0/16.

```

module "net_connections" {
    source          = "terraform-aws-modules/vpc/aws"
    version        = "~> 6.6.1"
    name           = "${var.project}-VPC"
    cidr           = "10.0.0.0/16"
    azs            = ["us-east-1a", "us-east-1b"]
    public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
    private_subnets = ["10.0.100.0/24", "10.0.200.0/24"]
    enable_nat_gateway = true
    single_nat_gateway = true
    enable_dns_hostnames = true
    enable_dns_support = true
}

```

- **CIDR /16:** provee mas de 65,000 direcciones IP, suficiente para escalar el proyecto con multiples servicios en el futuro.
- **2 zonas de disponibilidad (AZs)** con subnets publicas y privadas en cada una garantizan alta disponibilidad. Si una AZ falla, las tareas en la otra AZ continuan operando.
- **NAT Gateway unico:** las tareas en subnets privadas necesitan acceso a internet para hacer pull de imagenes ECR y enviar logs a CloudWatch. Un solo NAT Gateway reduce costos (~\$32/mes extra) respecto a uno por AZ.
- **Las subnets publicas** alojan el ALB (unico componente con acceso desde internet) mientras **las subnets privadas** alojan las tareas ECS, aislandolas de la red publica.

Security Groups

Se definen dos Security Groups con minimo privilegio:

ALB (publico):

```

module "security_group_alb" {
  ingress_rules = {
    http  = { from_port = 80, ip_protocol = "tcp", cidr_ipv4 = "0.0.0.0/0" }
    https = { from_port = 443, ip_protocol = "tcp", cidr_ipv4 = "0.0.0.0/0" }
  }
  egress_rules = {
    all = { ip_protocol = "-1", cidr_ipv4 = "0.0.0.0/0" }
  }
}

```

ECS (privado):

```

module "security_group_priv" {
  ingress_rules = {
    from-alb = {
      ip_protocol          = "-1"
      referenced_security_group_id = module.security_group_alb.id
    }
  }
  egress_rules = {
    all = { ip_protocol = "-1", cidr_ipv4 = "0.0.0.0/0" }
  }
}

```

- **Decision:** el SG privado solo acepta trafico desde el SG del ALB (no por IP, sino por referencia al SG). Esto significa que aunque alguien conozca la IP interna de las tareas ECS, no puede acceder directamente a menos que venga a traves del ALB. El egress esta abierto para permitir que los contenedores descarguen dependencias, envíen logs, y conecten con ECR/DynamoDB/S3 mediante los VPC endpoints implícitos de AWS.

Application Load Balancer

El ALB es el unico punto de entrada publico. Se configura con el modulo `terraform-aws-modules/alb/aws` ~> 10.5.0:

```

module "alb" {
  name      = "${var.project}-ALB"
  subnets = module.net_connections.public_subnets

  listeners = {
    ex-http = {
      port      = 80
      protocol  = "HTTP"
      forward   = { target_group_key = "web-fargate" }
      rules = {
        booking      = { priority = 10, conditions = [{ path_pattern = { values = ["/api/bo"]
        testimonios  = { priority = 20, conditions = [{ path_pattern = { values = ["/api/tes

```

```

        lugares      = { priority = 30, conditions = [{ path_pattern = { values = ["/api/lug
    }
  }
}
}

```

- **4 Target Groups:** web-fargate (tipo IP, para ECS), booking-lambda, testimonios-lambda, lugares-lambda (tipo Lambda).
- **Path-based routing:** el ALB inspecciona la URL y dirige:
 - /api/booking → Lambda de reservas
 - /api/testimonios* → Lambda de testimonios
 - /api/lugares* → Lambda de lugares
 - /* (default) → ECS Fargate (React SPA)
- **Decision:** se eligio ALB sobre API Gateway porque necesitabamos un unico DNS que sirviera tanto el SPA estatico como las APIs. El ruteo por path del ALB permite tener todo bajo un mismo dominio, simplificando CORS (todas las llamadas desde el frontend son al mismo origen /api/*). Ademàs, el ALB tiene menor latencia que API Gateway para trafico HTTP simple y su costo es predecible (\$0.0225/hora + \$0.008/LCU).

ECS Fargate (Frontend)

El frontend es una aplicacion React construida con Vite y servida por Nginx. El despliegue usa ECS Fargate con el modulo terraform-aws-modules/ecs/aws ~> 7.5.0:

```

module "ecs" {
  cluster_name = "${var.project}-CLUSTER"
  services = {
    web = {
      cpu           = 256
      memory        = 512
      desired_count = 2
      launch_type   = "FARGATE"
      subnet_ids    = module.net_connections.private_subnets
      assign_public_ip = false
      container_definitions = [
        web = {
          image = "${data.aws_ecr_repository.web.repository_url}:${var.ecr_image_tag}"
          portMappings = [{ containerPort = 80 }]
        }
      ]
    }
  }
}

```

Dockerfile multi-stage:

```

FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.27-alpine
ARG ALB_URL
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
RUN sed -i "s/__ALB_URL__/${ALB_URL}/g" /etc/nginx/conf.d/default.conf

```

- **Decision Fargate vs EC2:** Fargate elimina la gestion de servidores (no hay que parchear SO, instalar Docker, ni administrar capacity). Para un equipo pequeno y un sitio de trafico bajo, el sobre costo de Fargate (~\$8.8/tarea/mes) se justifica frente al costo operativo de mantener instancias EC2.
- **256 CPU / 512 MB:** combinacion minima valida de Fargate. El sitio es estatico (Nginx sirviendo HTML/JS/CSS), por lo que no requiere mas recursos.
- **2 tareas en 2 AZs:** garantiza que si una tarea o AZ falla, el ALB dirige trafico a la tarea sana. El `desired_count = 2` asegura disponibilidad sin disparar costos.
- **assign_public_ip = false:** las tareas corren en subnets privadas sin IP publica, accesibles solo a traves del ALB.
- **Nginx __ALB_URL__:** el placeholder se reemplaza en tiempo de build con el DNS real del ALB (obtenido via `terraform output -raw alb_dns`). Esto permite que el contenedor sepa a donde enviar las peticiones `/api/*` una vez dentro de la VPC.

Lambda (Backend serverless)

Se implementan 4 funciones Lambda en Python 3.12 usando el modulo `terraform-aws-modules/lambda/aws` ~> 8.8.0. Cada funcion tiene politicas IAM acotadas a los recursos minimos que necesita.

Lambda de lugares (/api/lugares*):

```

def handler(event, context):
    method = event.get("httpMethod", "GET")
    path = event.get("path", "/api/lugares")
    if method == "GET" and path == "/api/lugares":
        return list_all()
    if method == "POST" and path == "/api/lugares":
        return create(json.loads(event.get("body", "{}")))
    if method == "PUT" and path.startswith("/api/lugares/"):

```

```

        return update(lugar_id, json.loads(event.get("body", "{}")))
    if method == "DELETE" and path.startswith("/api/lugares/"):
        return delete(lugar_id)

```

- CRUD completo almacenando registros en DynamoDB e imagenes en S3.
- Las imagenes se reciben en base64 desde el panel admin y se decodifican antes de subirlas a S3.
- Al consultar (GET), la Lambda genera URLs prefirmadas de S3 con 1 hora de expiracion, evitando hacer el bucket publico.
- Se usa `Scan` con `FilterExpression` por `entity_type = "lugar"` para listar solo lugares, ya que la tabla es compartida.

Lambda de testimonios (/api/testimonios*):

- Almacena metadatos como JSON en `testimonios-meta/{uuid}.json` y las fotos en `testimonios/{uuid}.jpg` dentro del bucket S3.
- GET publico retorna solo testimonios con `status = "approved"`, ordenados por fecha.
- GET admin retorna todos los testimonios sin filtrar para moderacion.
- POST crea testimonio con `status = "pending"`.
- PUT permite aprobar (`status = "approved"`) o rechazar (`status = "rejected"`).
- DELETE elimina tanto la imagen como los metadatos de S3.

Decision S3 vs DynamoDB para testimonios: las fotos son binarias y pueden ser grandes (múltiples MB en base64). Almacenar metadata + fotos juntos en S3 mantiene los datos autocontenidos y evita el limite de 400KB por item de DynamoDB. Ademas, listar objetos en S3 es eficiente para pocos items (carga baja esperada).

Lambda de reservas (/api/booking):

```

# Guarda en DynamoDB y envia email via SES
item = {
    "id": str(uuid.uuid4()),
    "nombre": body["nombre"],
    "user-mail": body["email"],
    "tramo": body["tramo"],
    "fecha": body["fecha"],
    "created_at": datetime.now(timezone.utc).isoformat(),
}
table.put_item(Item=item)
ses.send_email(
    Source=os.environ["SENDER_EMAIL"],
    Destination={"ToAddresses": [body["email"]]},
    Message={...}
)

```

- Recibe POST con nombre, email, tramo, fecha. Almacena en DynamoDB

y envia confirmacion por email via SES.

- Tiene una Function URL publica como respaldo (sin autenticacion, CORS abierto).

Lambda de catalogo S3 (trigger: `s3:ObjectCreated:*`):

- Se ejecuta automaticamente cuando se sube un archivo al bucket S3.
- Lee los metadatos del objeto y escribe una entrada en DynamoDB con `id`, `s3_key`, `filename`, y `created_at`.
- Sirve como respaldo de catalogacion para debugging o auditoria.

DynamoDB

Se usa una unica tabla compartida para todas las entidades, diferenciadas por el atributo `entity_type`:

```
module "dynamodb-table" {
  name      = "ulagos-tin-lab3"
  hash_key  = "id"
  attributes = [
    { name = "id",          type = "S" },
    { name = "entity_type", type = "S" },
    { name = "recorrido",   type = "S" },
    { name = "user-mail",   type = "S" },
    { name = "tramo",       type = "S" },
    { name = "fecha",       type = "S" },
  ]
  global_secondary_indexes = [
    { name = "entity_type-index", hash_key = "entity_type" },
    { name = "user-mail-index",   hash_key = "user-mail" },
    { name = "tramo-index",       hash_key = "tramo" },
    { name = "fecha-index",      hash_key = "fecha" },
    { name = "recorrido-index",  hash_key = "recorrido" },
  ]
  server_side_encryption_enabled = true
}
```

- **Decision single-table vs multi-table:** una sola tabla con `entity_type` como discriminador simplifica la gestion (un solo recurso Terraform, un solo ARN en las politicas IAM) y reduce costos. Los GSI permiten consultar por dimensiones especificas (ej. filtrar lugares por `recorrido`, buscar reservas por `user-mail`).
- **Cifrado:** habilitado con KMS administrado por AWS sin costo adicional.
- **On-demand:** no se configura capacidad provisionada, ideal para carga baja y esporadica (trafico de un sitio universitario).

S3

Bucket unico ulagos-tin-lab3-bucket-us-east-1 para todas las imagenes y metadatos:

```
module "s3-bucket" {
  source = "terraform-aws-modules/s3-bucket/aws"
  version = "~> 5.14.1"
  bucket = "${lower(var.project)}-bucket-${var.region}"
}
```

- **Cifrado SSE-S3 (AES256)** activado por defecto.
- **URLs prefirmadas:** en vez de hacer el bucket publico, las Lambdas generan URLs temporales de 1 hora con `generate_presigned_url()`. Esto mantiene los datos privados pero accesibles desde el frontend.
- **Estructura de carpetas:** lugares/{uuid}.jpg, testimonios/{uuid}.jpg, testimonios-meta/{uuid}.json.

SES

```
module "ses" {
  source      = "cloudposse/ses/aws"
  domain      = "benhub.cl"
  verify_dkim = false
}
```

- Envia confirmaciones de reserva desde `antiturismo@benhub.cl`.
- El dominio `benhub.cl` esta verificado en SES con registros DKIM configurados en Cloudflare externamente.
- **Decision SES vs SNS:** SES es el servicio nativo de AWS para email transaccional. Para el volumen esperado (pocas reservas al dia), SES es esencialmente gratuito (62,000 emails/mes en free tier).

IAM

Rol de ejecucion ECS:

```
module "iam_role" {
  trust_policy_permissions = {
    ecstasks = {
      actions = ["sts:AssumeRole"]
      principals = [{ type = "Service", identifiers = ["ecs-tasks.amazonaws.com"] }]
    }
  }
  policies = { ecs_execution = module.iam_policy.arn }
}
```

Con politica acotada a: - `ecr:GetAuthorizationToken`, `ecr:BatchCheckLayerAvailability`, `ecr:GetDownloadUrlForLayer`, `ecr:BatchGetImage` (pull de imagenes) -

logs:CreateLogStream, logs:PutLogEvents (envio de logs a CloudWatch)

Roles Lambda: cada funcion tiene politicas inline que solo permiten exactamente lo que necesita:

Lambda	Permisos
lugares	dynamodb:PutItem/GetItem/UpdateItem/DeleteItem/Scan en tabla + indices, s3:PutObject/GetObject en bucket
testimonios	s3:PutObject/GetObject/ListBucket/DeleteObject en bucket (no usa DynamoDB)
booking	dynamodb:PutItem en tabla, ses:SendEmail/SendRawEmail en *
S3-trigger	dynamodb:PutItem/GetItem/Query, s3:PutObject/GetObject

- **Decision:** principios de minimo privilegio. Cada Lambda solo puede acceder a los recursos y operaciones que necesita. No hay politicas con * salvo SES (donde el ARN de identidad verificado es dinamico).

ECR

```
data "aws_ecr_repository" "web" {  
  name = "${lower(var.project)}-web"  
}
```

- El repositorio ECR se creo manualmente fuera de Terraform (**data source**, no **resource**). Esto evita que un **terraform destroy** accidental elimine todas las imagenes Docker.
- Las imagenes siguen versionado semantico: v1.0.0, v1.0.1, v1.0.2.
- El tag de imagen activo se controla desde **terraform.tfvars** con **ecr_image_tag**.

CI/CD (docker_build.sh)

Script de bash que automatiza el build y despliegue:

```
ALB_URL=$(terraform output -raw alb_dns)  
APP_TAG=$(grep -oP 'ecr_image_tag\s*=\s*\K[^\s]+' terraform.tfvars)  
  
docker build --no-cache -t $APP:$APP_TAG \  
  --build-arg ALB_URL="$ALB_URL" \  
  ../sitio-web-2/  
  
docker push $ECR_URI:$APP_TAG  
aws ecs update-service --cluster $CLUSTER --service $SERVICE --force-new-deployment
```

1. Lee el DNS del ALB desde `terraform output` (siempre actualizado).
2. Lee el tag de imagen desde `terraform.tfvars` (unica fuente de verdad).
3. Construye la imagen Docker con `--no-cache` para garantizar build fresco.
4. Prueba localmente con `docker run + curl` (smoke test).
5. Pushea a ECR y fuerza redeploy en ECS.
6. Luego `terraform apply` actualiza la task definition para apuntar al nuevo tag.

d) Analisis de costos

Estimacion mensual

Recurso	Configuracion	Costo unitario	Cantidad	Costo mensual (USD)
ECS	256 CPU +	CPU:	2 tareas	~\$17.6
Fargate	512 MB	\$0.04048/h,	× 730h	
(tareas)	RAM	RAM:		
		\$0.00445/GB/h		
ALB	1 balanceador activo	\$0.0225/hora	1 × 730h	\$16.2
ALB	Trafico bajo	\$0.008/LCU-hora	~1 LCU	~\$5.8
LCU	estimado			
NAT	1 NAT	\$0.045/hora	1 × 730h	~\$32.4
Gateway				
Lambda	4 funciones	\$0.20/1M req + Free Tier	< 1M req/mes	~\$0
DynamoDB	On-demand	\$1.25/1M WRU + \$0.25/1M RRU	Carga baja	~\$0
S3	~1 GB almacenado	\$0.023/GB/mes	1 GB	~\$0.02
CloudWatch	5 GB logs	\$0.50/GB	5 GB	\$2.5
Logs		ingesta		
ECR	~500 MB imagenes	\$0.10/GB/mes	0.5 GB	\$0.05
Total estimado				~\$75/mes

Comparacion con alternativa: EC2

Enfoque	Costo mensual	Mantenimiento	Escalabilidad
Fargate (actual)	~\$58/mes	Ninguno (AWS gestiona)	Automatica (auto-scaling)
EC2 t3.micro × 2	~\$30/mes (instancias) + \$16 ALB = \$46	SO, parches, Docker, seguridad	Manual (agregar instancias)

Analisis: Fargate cuesta aproximadamente \$12/mes mas que EC2 en este escenario, pero elimina completamente la carga operativa: no hay que mantener servidores Linux, actualizar paquetes de seguridad, ni monitorear capacidad. Para un equipo de 2 personas y un proyecto academico, el ahorro en tiempo de operacion justifica ampliamente la diferencia de costo.

Oportunidades de optimizacion

- **NAT Gateway:** es el componente mas caro (~\$32/mes). Podria eliminarse si ECS se moviera a subnets publicas, pero se perderia el aislamiento de red. Alternativa: usar VPC Endpoints para ECR y CloudWatch (gratis/trafico bajo), eliminando la necesidad del NAT.
- **Escalar a 1 tarea:** fuera del horario de la presentacion, `desired_count = 1` reduce el costo de Fargate a la mitad (~\$8.8/mes).
- **DynamoDB provisioned:** para carga predecible, capacidad provisionada (25 WCU/25 RCU) cuesta ~\$12/mes vs on-demand, pero on-demand es \$0 con trafico bajo.
- **Lambda:** el Free Tier de AWS incluye 1 millon de invocaciones gratis al mes, por lo que el backend serverless tiene costo \$0 para el volumen esperado.

e) Capturas

Panel Administrador

Panel Admin Lugares

Figura 2: Panel Admin Lugares

El panel permite crear, editar y eliminar lugares con imagenes. Cada lugar pertenece a un recorrido (ej: “Pasado”, “Marzo”) y tiene orden de aparicion.

Moderacion de Testimonios

Panel Admin Testimonios

Figura 3: Panel Admin Testimonios

El administrador puede aprobar, rechazar o eliminar testimonios enviados por usuarios. Solo los aprobados aparecen en el carrusel publico.

Sitio web publico

Sitio web AntiTurismo

Figura 4: Sitio web AntiTurismo

La landing page muestra el hero, lugares filtrados por recorrido, carrusel de testimonios aprobados, y formulario de reserva.

ECS corriendo

ECS Service Running

Figura 5: ECS Service Running

El servicio ECS mantiene 2 tareas en ejecucion sobre 2 zonas de disponibilidad. El ALB distribuye trafico entre ambas.

Repositorio ECR

ECR Repository

Figura 6: ECR Repository

Las imagenes Docker se versionan con tags semanticos (v1.0.0, v1.0.1, v1.0.2). Cada nuevo despliegue genera una imagen inmutable.

Lambda Functions

Las 4 funciones Lambda procesan APIs REST sin servidores. Cada una tiene politicas IAM acotadas a los recursos minimos necesarios.

f) Conclusiones

La arquitectura implementada combina contenedores (ECS Fargate) para el frontend y funciones serverless (Lambda) para el backend, logrando un sistema **end-to-end funcional, escalable y seguro** desplegado completamente en AWS.

Fortalezas de la solucion:

1. **Infraestructura comoCodigo:** cada recurso AWS esta definido en Terraform, permitiendo recrear el entorno completo en minutos con **terraform apply**. El estado remoto en S3 permite colaboracion entre desarrolladores.

Funciones Lambda

Figura 7: Funciones Lambda

2. **Alta disponibilidad:** los componentes criticos (ALB, ECS, DynamoDB) operan en al menos 2 zonas de disponibilidad. Si una AZ falla, el servicio continua funcionando.
3. **Seguridad por capas:** los recursos de computo (ECS, Lambda) no estan expuestos directamente a internet. Solo el ALB tiene acceso publico, y las tareas ECS corren en subnets privadas sin IP publica.
4. **Serverless donde aplica:** las APIs usan Lambda (pago por uso, escala a cero), mientras que el frontend usa contenedores (necesario para servir el SPA con Nginx). Esta combinacion optimiza costos para la carga esperada.
5. **Minimo privilegio IAM:** cada Lambda y el rol de ejecucion ECS tienen permisos acotados a exactamente lo que necesitan. No hay politicas con ***** indiscriminado.

Mejoras futuras:

- Agregar HTTPS mediante certificado ACM y listener en puerto 443.
- Migrar autenticacion del panel admin al backend (JWT o API Key en Lambda) para eliminar la validacion client-side.
- Implementar CI/CD con GitHub Actions para construir y desplegar automaticamente al hacer push a **main**.
- Agregar CloudFront como CDN para los assets estaticos, reduciendo latencia y carga al ALB.
- Activar auto-scaling de ECS (`enable_autoscaling = true`) para manejar picos de trafico.